GUIDE DE PRÉPARATION · ODOO 19



Odoo : flux de travail et bases du développement

Un petit voyage visuel pour comprendre ce que fait une entreprise, puis comment Odoo le traduit en code.

🔁 Flux ERP · 🐍 Python / ORM · 🧩 Vues XML

PRÉPARÉ PAR
Ouchari Ibrahim

Cours ERP · ISID
Aucun prérequis : on part de zéro

AVANT DE COMMENCER

Votre carte du voyage

Aucune connaissance préalable n'est nécessaire. Le guide a deux parties : d'abord comprendre ce que fait l'entreprise, ensuite comprendre comment Odoo est codé. Chaque section a son code (A1, B3...) et tient sur une ou deux pages.

Ⓐ PARTIE 1 · Fonctionnel

Comprendre le travail d'une entreprise : ventes, achats, stock, factures.

- **A1** — **Le circuit de vente** — CRM → Ventes → Stock → Facture
- **A2** — **Le circuit d'achat** — Demande de prix → Réception → Facture
- **A3** — **Documents, modèles et fuites** — Les noms techniques et l'argent perdu

Ⓑ PARTIE 2 · Développement

Comprendre comment Odoo est écrit : Python, modèles, droits, vues.

- **B1** — **Rappel Python** — classes, self, dict, f-string
- **B2** — **Un module Odoo** — la boîte qu'on installe
- **B3** — **Modèles et champs** — les tables d'Odoo
- **B4** — **Droits d'accès** — qui a le droit de quoi
- **B5** — **Calculs et règles métier** — depends, constrains, onchange
- **B6** — **Surcharger create() et write()** — create, write, super
- **B7** — **Chercher des données** — search et domaines
- **B8** — **Les vues** — XML, XPath, menus
- **B9** — **Mini-projet** — tout assembler

🧭 Comment lire ?

À RETENIR
L'idée à garder.

ANALOGIE
Une image de la vie réelle.

PIÈGE
Une erreur classique.

ESSAYEZ
Un geste à faire dans Odoo.

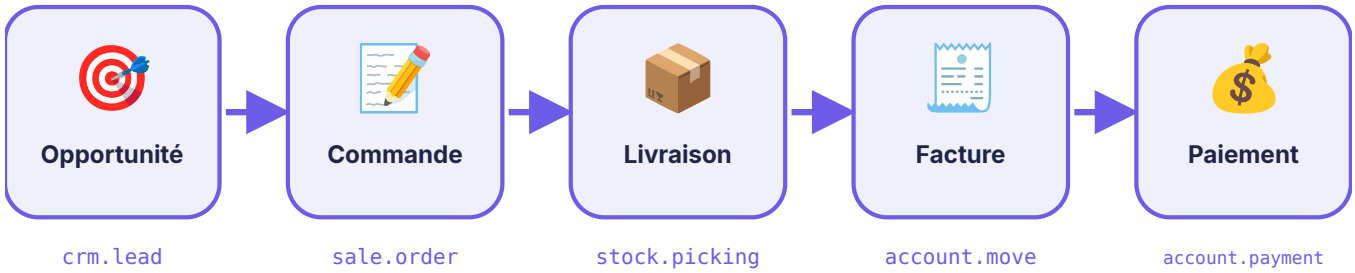
PARTIE 1 · FONCTIONNEL

A1 Le circuit de vente

De la première prise de contact jusqu'au paiement du client

ANALOGIE : LE RESTAURANT

Le client commande (**Ventes**), la cuisine prépare (**Stock**), la caisse encaisse (**Facturation**). Si la cuisine sert sans prévenir la caisse, le restaurant perd de l'argent. Odoo relie toutes ces étapes pour que personne n'oublie rien.



Étape par étape

1. **Une opportunité arrive (module CRM)**
   Un prospect s'intéresse à vos produits. On le suit dans un pipeline visuel.
   📍 CRM → Pipeline
2. **On envoie un devis**
   Depuis l'opportunité, **Nouveau devis** : le client est déjà rempli. Odoo calcule les totaux.
   📍 Ventes → Devis
3. **On confirme : le devis devient commande**
   Odoo crée **tout seul** le bon de livraison dans Stock. Personne ne ressaisit rien.
   📍 Bouton Confirmer
4. **On livre**
   L'entrepôt valide la livraison : le stock diminue et la commande devient « à facturer ».
   📍 Stock → Transferts
5. **On facture, puis le client paie**
   Créer la facture depuis la commande. Le paiement est rapproché de la facture.
   📍 Facturation → Clients

ESSAYEZ

Dans Odoo : **Ventes → Devis → Nouveau**, ajoutez un produit, cliquez **Confirmer**. Un bouton **Livraison** apparaît en haut : le circuit s'est activé.

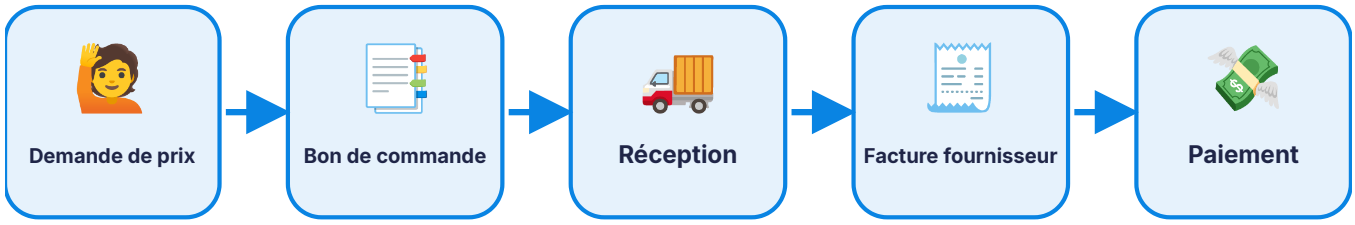
PARTIE 1 · FONCTIONNEL

A2 Le circuit d'achat

De la demande au fournisseur jusqu'au paiement

ANALOGIE : LE MAGASIN QUI SE RÉAPPROVISIONNE

Le gérant voit que les étagères se vident. Il demande un prix au grossiste, commande, reçoit les cartons, vérifie, puis paie la facture. Odoo suit chacun de ces gestes.

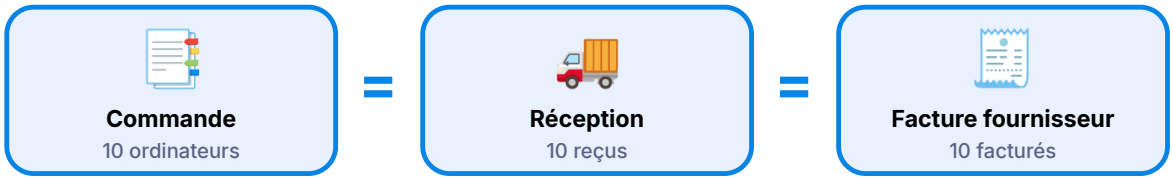


`purchase.order` `purchase.order` `stock.picking` `account.move` `account.payment`

Étape par étape

1. **Demande de prix**
   On choisit le fournisseur et les produits. Odoo peut envoyer la demande par e-mail.
   📍 Achats → Commandes → Nouveau
2. **On confirme : bon de commande**
   Le document devient officiel et une **réception en attente** apparaît automatiquement dans Stock.
   📍 Bouton Confirmer la commande
3. **On réceptionne**
   Quand les cartons arrivent, on valide les quantités vraiment reçues : le stock augmente.
   📍 Stock → Réceptions
4. **Facture fournisseur**
   Depuis la commande, on crée la facture du fournisseur et on la compare à la facture papier.
   📍 Bouton Créer la facture
5. **Paiement**
   À l'échéance, on enregistre le paiement. La dette est soldée.

🔍 Le rapprochement à trois voies



✔ **Les trois doivent correspondre avant de payer**

À RETENIR

Avant de payer, le comptable compare **la commande, la réception et la facture**. Si une seule diffère (quantité, prix), il y a un problème à régler avant de payer.

PARTIE 1 · FONCTIONNEL

A3 Documents, modèles et fuites

Chaque document a un nom technique, et chaque maillon cassé fait perdre de l'argent

🏷️ Le nom technique de chaque document

Dans le code, un type de document s'appelle un **modèle**. Voici la traduction :

Dans la vraie vie	Dans Odoo (nom du modèle)
Un client ou un fournisseur	`res.partner`
Un produit	`product.template` / `product.product`
Une opportunité commerciale	`crm.lead`
Un devis ou une commande client (et ses lignes)	`sale.order` / `sale.order.line`
Une livraison ou une réception	`stock.picking`
Une facture (client ou fournisseur)	`account.move`

🕳️ Où l'argent s'échappe

Quand un maillon du circuit casse, l'entreprise perd de l'argent *sans s'en rendre compte*. Trois fuites typiques :



Livré mais pas facturé

Le client a reçu sa marchandise, mais personne n'a envoyé la facture.



Stock qui dort

Des produits achetés qui ne se vendent plus : de l'argent bloqué sur une étagère.



Remise trop généreuse

Une vente faite à perte à cause d'un rabais oublié.

🔑 Deux champs à connaître sur une commande client

Champ	Valeurs principales	Ce que ça dit
`state`	`draft` → `sale` → `cancel`	Devis, commande confirmée, annulée
`invoice_status`	`no` · `to invoice` · `invoiced`	Rien à facturer · à facturer · déjà facturé

À RETENIR

Repérer ces fuites est un vrai besoin des entreprises. Mais avant de pouvoir les détecter, il faut d'abord **bien comprendre le circuit** et le nom technique de chaque document.

PARTIE 2 · DÉVELOPPEMENT

B1 Rappel Python

Les cinq outils Python qu'on retrouvera dans tout le code d'Odoo

ANALOGIE : LE MOULE À GÂTEAUX

Une **classe** est un moule. Chaque gâteau fabriqué avec ce moule est un **objet**. Et `self`, c'est « ce gâteau-ci » : l'objet sur lequel on travaille en ce moment.

1 · CLASSE ET SELF

```
class Order:                        # le moule
    def __init__(self, ref, total):
        self.ref = ref              # self = cette commande
        self.total = total

    def label(self):
        return f"{self.ref}: {self.total} TND"

o = Order('SO42', 4200)
print(o.label())                    # SO42: 4200 TND
```

2 · DICTIONNAIRE, LISTE ET BOUCLE

```
vals = {'name': 'Alert', 'amount': 500}   # dict
print(vals['amount'])                     # 500
vals['state'] = 'new'                     # add a key

orders = ['SO1', 'SO2', 'SO3']            # list
for ref in orders:
    print(ref)
```

3 · F-STRING (TEXTE AVEC VARIABLES)

```
name = 'Alert'
amount = 500
title = f"{name} : {amount}"
print(title)        # Alert : 500
```

4 · CONDITIONS (L'INDENTATION COMPTE !)

```
if amount >= 1000:
    level = 'medium'
elif amount >= 100:
    level = 'low'
else:
    level = 'none'
```

5 · LAMBDA

```
is_big = lambda o: o.total > 5000
# = une mini-fonction sans nom
```

En Python	Dans Odoo, ça sert à…
`class`	décrire un modèle (une « table »)
`self`	désigner les enregistrements sur lesquels la méthode travaille
dictionnaire `{…}`	transporter les valeurs à enregistrer (`vals`)
`for … in self`	traiter les enregistrements un par un
`f"…"`	fabriquer des noms et des messages
`lambda`	écrire un filtre rapide dans `filtered`

À RETENIR

Les conditions sont lues **de haut en bas** : la première qui est vraie gagne, et les suivantes sont ignorées.

PIÈGE

En Python, ce sont les **espaces au début de la ligne** (l'indentation, 4 espaces) qui disent ce qui est dans le `if` ou dans la boucle. Un mauvais décalage change tout le sens du code.

PARTIE 2 · DÉVELOPPEMENT

B2 Un module Odoo

La boîte qu'on installe pour ajouter une fonctionnalité

ANALOGIE : UNE BOÎTE DE LEGO

Un module est une boîte avec une étiquette (le manifeste). À l'intérieur : des pièces de données (Python), d'écrans (XML) et de droits d'accès (CSV). On l'installe, et Odoo l'assemble tout seul.

📁 La structure d'un module

DOSSIER DU MODULE

```
risk_demo/
├── __init__.py          # importe models
├── __manifest__.py      # la carte d'identité
├── models/
│   ├── __init__.py
│   ├── risk_alert.py
│   └── sale_order.py
├── views/
│   ├── risk_alert_views.xml
│   └── sale_order_views.xml
└── security/
    └── ir.model.access.csv # qui a le droit
```

Dossier	Contenu
`models/`	Le Python : les données et la logique
`views/`	Le XML : ce que l'utilisateur voit
`security/`	Les droits d'accès

🪪 Le fichier manifeste

__MANIFEST__.PY

```
{
    'name': 'Risk Demo',
    'version': '19.0.1.0.0',
    'summary': 'Detect unbilled orders',
    'depends': ['sale_stock'],
    'data': [
        'security/ir.model.access.csv',
        'views/risk_alert_views.xml',
        'views/sale_order_views.xml',
    ],
    'license': 'LGPL-3',
    'application': True,
}
```

`depends` : les modules à installer avant le nôtre.
`data` : les fichiers XML/CSV chargés à l'installation, **dans cet ordre** (sécurité d'abord, vues ensuite).

PIÈGE

Un fichier XML qui n'est **pas listé dans** `data`, ou un oubli dans un `__init__.py` : Odoo ne dit rien... et rien n'apparaît à l'écran.

À RETENIR

Installer = charger le module la première fois. **Mettre à jour** (Applications → Mettre à jour) = recharger après avoir modifié le code ou les vues.

PARTIE 2 · DÉVELOPPEMENT

B3 Modèles et champs

Comment Odoo range les informations

ANALOGIE : EXCEL

Un **modèle** Odoo, c'est comme une feuille Excel : chaque colonne a un nom et un type. Odoo crée la vraie table dans la base de données à votre place (`risk.alert` devient la table `risk_alert`).

📊 Un modèle ressemble à un tableau

name (Char)	amount_at_risk (Float)	order_id (Many2one)	state (Selection)
Unbilled SO0042	4 200.0	SO0042	new
Unbilled SO0057	980.0	SO0057	approved

Le **modèle** = le tableau · un **champ** = une colonne · un **enregistrement** = une ligne

🧱 Les types de champs les plus courants

🔤 **Char**
texte court

📄 **Text**
texte long

🔢 **Integer / Float**
nombres

✅ **Boolean**
vrai / faux

📅 **Date / Datetime**
dates

🔽 **Selection**
liste de choix

🔗 **Many2one**
lien vers 1

🧺 **One2many**
liste de liens

MODELS/RISK_ALERT.PY

```
from odoo import models, fields


class RiskAlert(models.Model):
    _name = 'risk.alert'
    _description = 'Risk Alert'

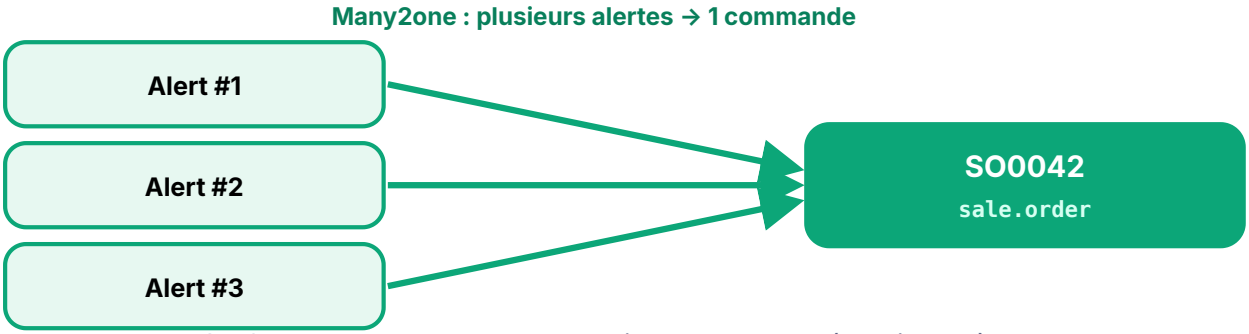
    name = fields.Char(string='Title', required=True)
    amount_at_risk = fields.Float(string='Amount at risk')
    order_id = fields.Many2one('sale.order', string='Sale order')
    state = fields.Selection(
        [('new', 'New'), ('approved', 'Approved'), ('done', 'Done')],
        default='new')
    closed_date = fields.Datetime()
    note = fields.Text()
```

PARTIE 2 · DÉVELOPPEMENT

B3 Modèles et champs

Relier des modèles, et étendre ceux d'Odoo

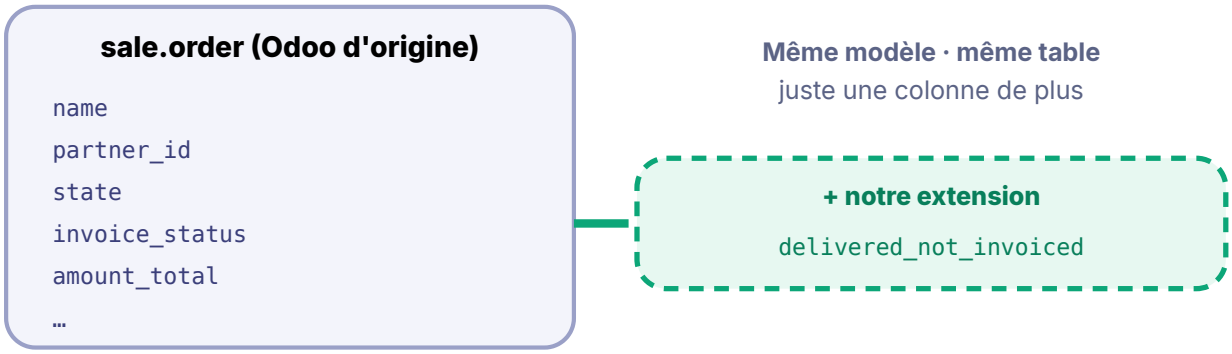
🔗 Relier deux modèles




```
ONE2MANY : LA LISTE INVERSE (LE 2E ARGUMENT = LE CHAMP MANY2ONE EN FACE)
alert_ids = fields.One2many('risk.alert', 'order_id')   # sur sale.order : le sens inverse
```

🧬 Étendre un modèle existant : `_inherit`

On n'a pas le droit (ni besoin) de modifier le code d'Odoo. À la place, on **ajoute** nos champs par-dessus.




```
MODELS/SALE_ORDER.PY
class SaleOrder(models.Model):
    _inherit = 'sale.order'      # pas de _name : on étend l'existant

    delivered_not_invoiced = fields.Boolean(
        string='Delivered, not invoiced')
```

À RETENIR

`_name` = **créer** un nouveau modèle.
`_inherit` seul = **étendre** un modèle existant : l'original n'est pas remplacé, il reçoit de nouveaux champs.

PIÈGE

Écrire `_name = 'sale.order'` au lieu de `_inherit` : vous essayez de recréer le modèle au lieu de l'étendre.

PARTIE 2 · DÉVELOPPEMENT

B4 Droits d'accès

Qui a le droit de voir, modifier, créer ou supprimer quoi ?

ANALOGIE : LA CLÉ DE LA PORTE

Vous venez de créer un modèle : c'est une nouvelle pièce dans la maison. Mais la porte est fermée à clé. Le fichier `ir.model.access.csv` distribue les clés et dit ce que chacun peut faire dans la pièce.



Sans règle, la porte reste fermée pour les utilisateurs normaux

📄 Une règle = une ligne dans un fichier CSV

SECURITY/IR.MODEL.ACCESS.CSV

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_risk_alert_user,risk.alert.user,model_risk_alert,base.group_user,1,1,1,0
```

Colonne	Ce qu'elle veut dire
`id`	Le nom unique de la règle (au choix, mais sans doublon)
`name`	Une description lisible
`model_id:id`	Le modèle concerné : `model_` + son nom avec des « _ » à la place des points
`group_id:id`	**Qui** reçoit la clé. Ici : tous les utilisateurs internes (on ne détaille pas les groupes aujourd'hui)
`perm_…`	Quatre interrupteurs : **1 = autorisé**, **0 = interdit**

🎚️ Les quatre interrupteurs de la ligne ci-dessus

1	1	1	0
perm_read	perm_write	perm_create	perm_unlink
voir	modifier	créer	supprimer

À RETENIR

Pas de règle = les utilisateurs normaux n'ont **aucun accès** au modèle. Le fichier se liste dans `data` du manifeste, **avant les vues**.

PIÈGE

En administrateur, tout semble marcher. L'erreur n'apparaît que quand un **autre utilisateur** essaie : « Vous n'avez pas le droit d'accéder… ».

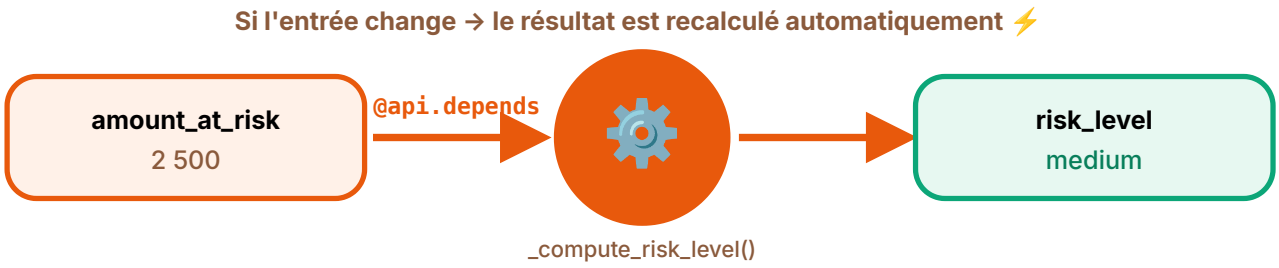
PARTIE 2 · DÉVELOPPEMENT

B5 Calculs et règles métier

Des champs qui se calculent seuls : comme une formule Excel

ANALOGIE : LA FORMULE EXCEL

Dans Excel, une cellule `=B2*C2` se met à jour quand B2 ou C2 change. Un **champ calculé** Odoo fait pareil : on lui dit de quels champs il dépend avec `@api.depends`.



MODELS/RISK_ALERT.PY

```
risk_level = fields.Selection(
    [('low', 'Low'), ('medium', 'Medium'), ('high', 'High')],
    compute='_compute_risk_level', store=True)

@api.depends('amount_at_risk')
def _compute_risk_level(self):
    for rec in self:
        if rec.amount_at_risk >= 10000:
            rec.risk_level = 'high'
        elif rec.amount_at_risk >= 1000:
            rec.risk_level = 'medium'
        else:
            rec.risk_level = 'low'
```

À RETENIR

`compute='_nom'` relie le champ à sa méthode. `@api.depends(...)` liste les champs qui déclenchent le recalcul.

`store=True` : la valeur est **enregistrée en base** (on peut chercher et trier dessus). Sans `store`, elle est recalculée à la volée.

PIÈGE

Toujours **boucler sur `self`** (`for rec in self`) et **donner une valeur au champ dans tous les cas** (if / elif / else). Sinon Odoo lève une erreur.

PARTIE 2 · DÉVELOPPEMENT

B5 Calculs et règles métier

Interdire le faux, aider l'utilisateur

🛑 Bloquer une donnée invalide

Une **contrainte** vérifie une règle métier à l'enregistrement. Si la règle est violée, on lève une `ValidationError` : la sauvegarde est refusée et l'utilisateur voit le message.

MODELS/RISK_ALERT.PY

```python
from odoo import api
from odoo.exceptions import ValidationError

@api.constrains('amount_at_risk')
def _check_amount(self):
    for rec in self:
        if rec.amount_at_risk < 0:
            raise ValidationError("The amount cannot be negative.")
```

💡 Aider pendant la saisie

MODELS/RISK_ALERT.PY

```python
@api.onchange('order_id')
def _onchange_order_id(self):
    if self.order_id:
        self.name = f"Check {self.order_id.name}"   # simple suggestion
```

⚖️ Les trois décorateurs côte à côte

	@api.depends	@api.constrains	@api.onchange
Rôle	Calculer une valeur	Interdire des données invalides	Aider l'utilisateur à remplir le formulaire
Quand ?	Quand les champs listés changent	À l'enregistrement (sauvegarde)	Pendant la saisie, *avant* de sauvegarder
Résultat	Valeur du champ mise à jour	Sauvegarde **bloquée** + message	Valeur suggérée, **aucun blocage**

🎮 Quel décorateur choisir ?

Situation
« Le niveau de risque dépend du montant »
@api.depends

Situation
« Un montant négatif est interdit »
@api.constrains

Situation
« Pré-remplir le titre quand on choisit une commande »
@api.onchange

PIÈGE

`@api.onchange` ne protège **rien** : il n'agit que dans le formulaire. Pour une vraie règle (qui doit tenir même avec un import ou du code), il faut `@api.constrains` .

PARTIE 2 · DÉVELOPPEMENT

B6 Surcharger create() et write()

Ajouter votre logique au moment où on crée ou modifie un enregistrement

ANALOGIE : LE SANDWICH

Surcharger une méthode, c'est faire un sandwich : votre code en **pain du dessus**, le code d'Odoo (`super()`) en **garniture**, et éventuellement du pain en dessous.



➕ `create()` : à la création

```
CREATE
@api.model_create_multi
def create(self, vals_list):
    for vals in vals_list:
        if not vals.get('name'):
            vals['name'] = 'New alert'
    return super().create(vals_list)
```

✏️ `write()` : à la modification

```
WRITE
def write(self, vals):
    if vals.get('state') == 'done':
        vals['closed_date'] = fields.Datetime.now()
    return super().write(vals)
```

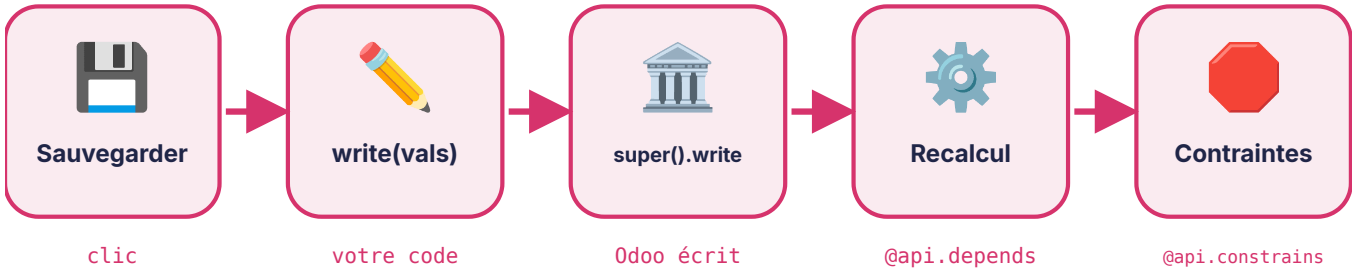
C'EST QUOI « VALS » ?

Un **dictionnaire** `{champ: valeur}` : pour `write` , ce qui est en train de changer. Pour `create` , une liste de dictionnaires (un par enregistrement).

POURQUOI SUPER() ?

Pour **appeler l'implémentation parente** : sans elle, rien n'est réellement enregistré.

⏱️ Ce qui se passe quand on clique « Sauvegarder »



PIÈGE

Oublier `super()` (ou oublier le `return`) : l'enregistrement n'est pas correctement créé ou modifié, et souvent aucune erreur ne vous prévient. C'est l'erreur n°1 des débutants.

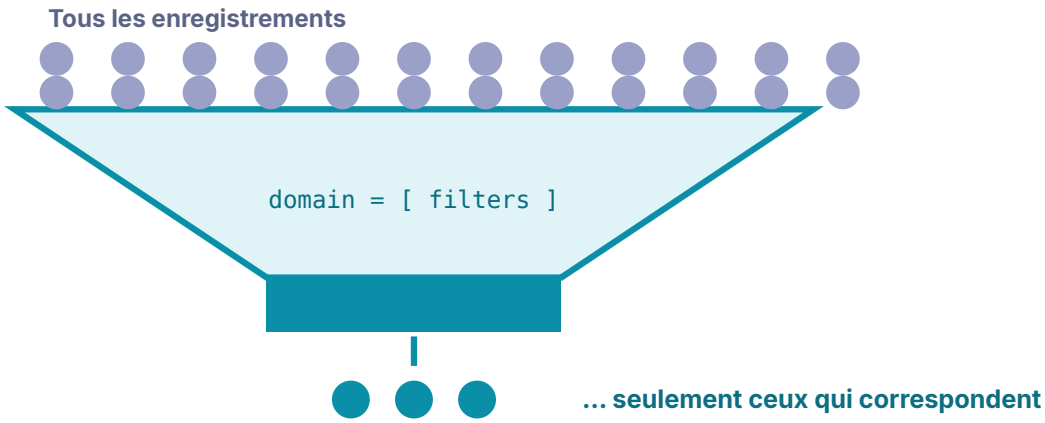
PARTIE 2 · DÉVELOPPEMENT

B7 Chercher des données

Interroger la base d'Odoo depuis Python

ANALOGIE : UN FILTRE EXCEL

Chercher dans Odoo, c'est appliquer un **filtre** sur un tableau. Le filtre s'appelle un **domaine** ; le résultat, un **recordset**.



🔍 Les outils de recherche

```python
Order = self.env['sale.order']          # le « tableau » des commandes

orders = Order.search([('state', '=', 'sale'),
                       ('invoice_status', '=', 'to invoice')])

n      = Order.search_count([('state', '=', 'sale')])    # combien ?
total  = sum(orders.mapped('amount_total'))              # somme d'une colonne
names  = orders.mapped('name')                           # liste des noms
big    = orders.filtered(lambda o: o.amount_total > 5000) # filtre en Python
new    = self.env['risk.alert'].create({'name': 'Alert', 'amount_at_risk': 500})
```

Un domaine = une liste de conditions

Condition	Sens
`('state', '=', 'sale')`	commande confirmée
`('amount_total', '>', 5000)`	montant > 5000
`('name', 'ilike', 'SO')`	contient « so »
`('state', 'in', ['draft','sent'])`	l'un des deux

Plusieurs conditions dans la liste = **ET**. Pour un **OU**, on met `'|'` devant (notation préfixe).

Un recordset = une collection d'enregistrements

Le résultat d'un `search` n'est pas une simple liste : c'est un **recordset**. On peut le parcourir, compter, filtrer et en extraire des colonnes.

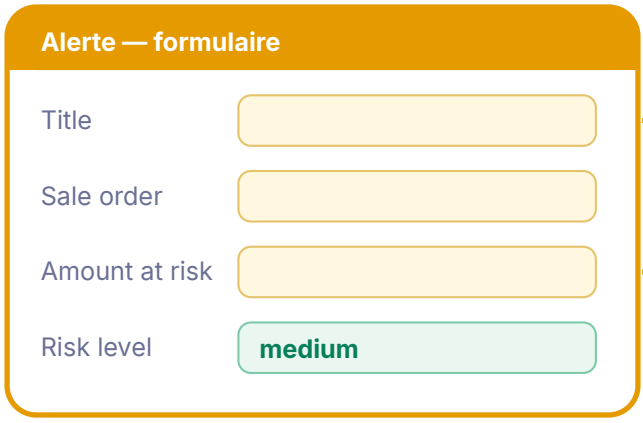
```python
for order in orders:
    print(order.name, order.partner_id.name)
```

B8 Les vues

Un fichier XML décrit chaque écran

ANALOGIE : LE PLAN D'UNE MAISON

Le **modèle** est la structure (les pièces), la **vue** est le plan d'affichage (quelles pièces on montre et où). Un même modèle peut avoir plusieurs vues : **form** (fiche), **list** (tableau), **kanban** (cartes)…




```xml
<form> ← l'écran
  <group> ← une zone
    <field name="name"/>
    <field name="order_id"/>
    <field name="amount_at_risk"/>
    <field name="risk_level"/>
  </group>
</form>
```

VIEWS/RISK_ALERT_VIEWS.XML

```xml
<record id="alert_view_form" model="ir.ui.view">
    <field name="name">risk.alert.form</field>
    <field name="model">risk.alert</field>
    <field name="arch" type="xml">
        <form>
            <sheet>
                <group>
                    <field name="name"/>
                    <field name="order_id"/>
                    <field name="amount_at_risk"/>
                    <field name="risk_level"/>
                </group>
            </sheet>
        </form>
    </field>
</record>
```

LA VUE LISTE (TABLEAU)

```xml
<!-- dans le même <record> -->
<list>
    <field name="name"/>
    <field name="amount_at_risk"/>
    <field name="risk_level"/>
</list>
```

ACTION + MENU

```xml
<record id="alert_action" model="ir.actions.act_window">
    <field name="name">Alerts</field>
    <field name="res_model">risk.alert</field>
    <field name="view_mode">list,form</field>
</record>

<menuitem id="alert_menu_root" name="Alerts"
          action="alert_action" sequence="10"/>
```

PARTIE 2 · DÉVELOPPEMENT

B8 Les vues

Modifier un écran d'Odoo sans toucher à son code : l'héritage de vue

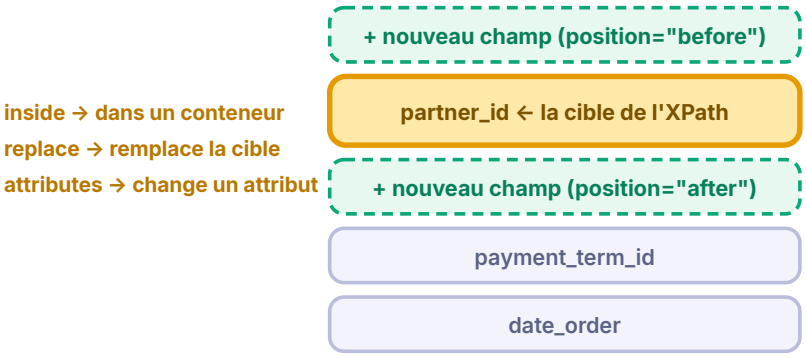
Comme pour les modèles, on ne modifie jamais l'écran d'origine : on le **cible** avec une vue héritière. Deux ingrédients :

`inherit_id`

Indique **quelle vue d'origine** on veut modifier (son identifiant externe, ex : `sale.view_order_form`).

XPath

Une « adresse » qui **localise un élément** dans le XML de la vue (ex : le champ `partner_id`).



On vise un élément de la vue d'origine, puis on dit où placer le nouveau

VIEWS/SALE_ORDER_VIEWS.XML

```xml
<record id="sale_order_view_form_risk" model="ir.ui.view">
    <field name="name">sale.order.form.risk</field>
    <field name="model">sale.order</field>
    <field name="inherit_id" ref="sale.view_order_form"/>
    <field name="arch" type="xml">
        <xpath expr="//field[@name='partner_id']" position="after">
            <field name="delivered_not_invoiced"/>
        </xpath>
    </field>
</record>
```

📍 Les positions

position	Effet
`after`	Place le nouveau contenu **juste en dessous / après** la cible
`before`	Place le nouveau contenu **avant** la cible
`inside`	Insère le contenu **à l'intérieur** de l'élément ciblé (à la fin)
`replace`	**Remplace** l'élément ciblé
`attributes`	Modifie les **attributs** de la cible (ex : `invisible`)

PIÈGE

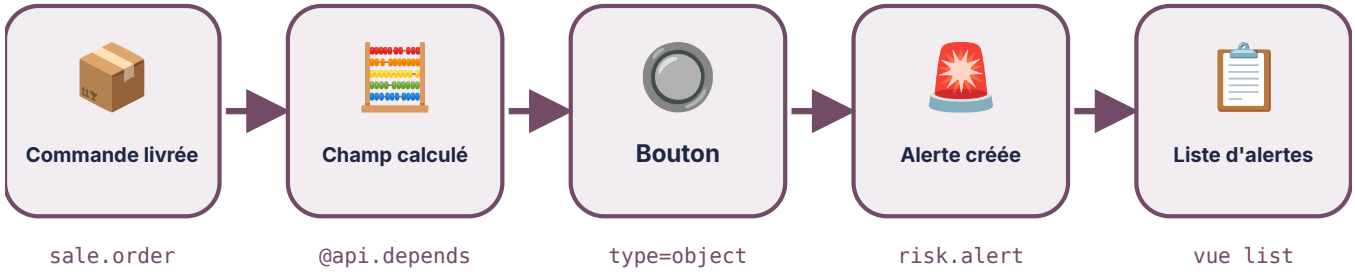
Si l'identifiant dans `ref="…"` ou l'expression XPath est faux, Odoo refuse d'installer la vue (erreur à l'installation). Vérifiez toujours le nom exact de la vue d'origine et du champ visé.

PARTIE 2 · DÉVELOPPEMENT

B9 Mini-projet

Assembler toutes les pièces en un vrai petit module

Chaque lundi, la direction demande : « *Quelles commandes avons-nous livrées sans les facturer ?* »
Voici un petit cas réel, que nous allons construire pièce par pièce.



🪪 1. Manifeste

`depends: sale_stock` pour accéder aux livraisons de la commande.

🧬 2. Extension

`_inherit` sur `sale.order` + champ calculé.

🚨 3. Modèle + vues

`risk.alert`, ses droits, ses vues, un menu, et un bouton hérité.

MODELS/SALE_ORDER.PY (1/2)

```python
class SaleOrder(models.Model):
    _inherit = 'sale.order'

    delivered_not_invoiced = fields.Boolean(
        compute='_compute_delivered_not_invoiced', store=True)

    @api.depends('invoice_status', 'picking_ids.state')
    def _compute_delivered_not_invoiced(self):
        for order in self:
            pickings = order.picking_ids.filtered(lambda p: p.state != 'cancel')
            order.delivered_not_invoiced = (
                bool(pickings)
                and all(p.state == 'done' for p in pickings)
                and order.invoice_status == 'to invoice')
```

MODELS/SALE_ORDER.PY (2/2)

```python
    def action_create_alert(self):
        for order in self.filtered('delivered_not_invoiced'):
            self.env['risk.alert'].create({
                'name': f"Delivered but not invoiced: {order.name}",
                'order_id': order.id,
                'amount_at_risk': order.amount_total,
            })
```

PARTIE 2 · DÉVELOPPEMENT

B9 Mini-projet

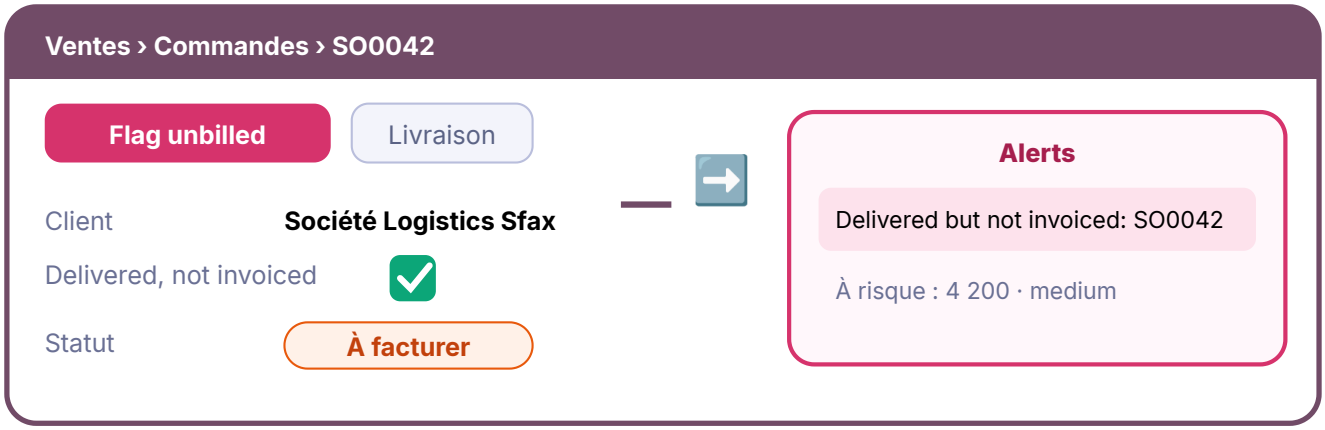
Le bouton, et ce que l'on voit à l'écran

🔘 Ajouter le bouton dans le formulaire de commande

VIEWS/SALE_ORDER_VIEWS.XML (À AJOUTER DANS L'HÉRITAGE)

```
<xpath expr="//header" position="inside">
    <button name="action_create_alert" type="object"
            string="Flag unbilled"
            invisible="not delivered_not_invoiced"/>
</xpath>
```

🖥️ Résultat attendu



🧩 Les briques utilisées

Brique	Section du guide
`for` , `lambda` , f-string, dictionnaire	B1 · Rappel Python
Manifeste, `depends` , `data`	B2 · Un module Odoo
`_inherit` , `Many2one`	B3 · Modèles et champs
`ir.model.access.csv`	B4 · Droits d'accès
`@api.depends` , `store=True`	B5 · Calculs et règles métier
`filtered` , `create` , recordsets	B7 · Chercher des données
XPath `position` , `inherit_id` , menu, action	B8 · Les vues

BRAVO

Vous venez de lire un petit module Odoo complet : un modèle, une extension, une règle de calcul, des droits et des vues. Au prochain cours, nous referons ce chemin ensemble, en codant chaque pièce.

FICHE RÉCAPITULATIVE

Tout tient sur une page

Gardez cette fiche sous la main : elle résume toutes les sections.

A1 · Le circuit de vente

CRM → Ventes → Stock → Facture → Paiement. Confirmer une commande crée la livraison toute seule.

A2 · Le circuit d'achat

Demande de prix → Bon de commande → Réception → Facture fournisseur. Rapprochement à trois voies : commande, réception, facture.

A3 · Documents, modèles et fuites

`sale.order` commande, `stock.picking` livraison, `account.move` facture. Fuites : livré non facturé, stock dormant, remise excessive.

B1 · Rappel Python

`class` moule, `self` « cet objet », dictionnaire `{…}`, f-string `f"…"`, `lambda` mini-fonction.

B2 · Un module Odoo

`__manifest__.py` (carte d'identité : `depends`, `data`) + `models/` + `views/` + `security/`.

B3 · Modèles et champs

`_name` crée · `_inherit` seul étend. Champs : `Char`, `Float`, `Selection`, `Many2one`…

B4 · Droits d'accès

`ir.model.access.csv` : une ligne par règle ; 4 interrupteurs `perm_…` (1 = oui, 0 = non). Sans règle, pas d'accès.

B5 · Calculs et règles métier

`@api.depends` calcule · `@api.constrains` bloque (ValidationError) · `@api.onchange` aide dans le formulaire.

B6 · Surcharger create() et write()

`create(vals_list)` / `write(vals)` + toujours `return super()…`

B7 · Chercher des données

`search`, `search_count`, `mapped`, `filtered`. Domaine = liste de `(champ, opérateur, valeur)`.

B8 · Les vues

`<form>`, `<list>`, action + menu. Héritage : `inherit_id` + XPath + `position` (after, before, inside, replace).

B9 · Mini-projet

Extension d'un modèle Odoo + modèle perso + droits + vues + bouton = un vrai module.

ET APRÈS ?

Au prochain cours, nous coderons un module complet depuis zéro. Grâce à ce guide, vous saurez déjà lire chaque ligne.

BONNE CHANCE

Bonne chance pour votre examen ! 🍀

BONUS · FACULTATIF

Essayez Odoo 19 en ligne

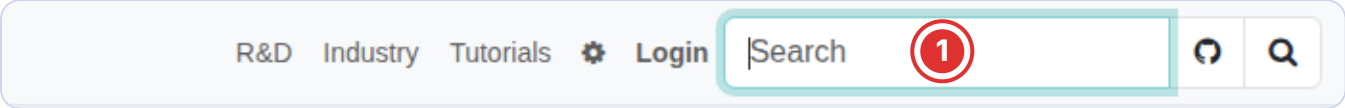
Sans rien installer sur votre ordinateur

Envie de manipuler Odoo vous-même ? Utilisez **Runbot**, la plateforme de test de l'équipe Odoo : elle propose des instances prêtes à l'emploi pour chaque version. Pour notre cours, c'est mieux que l'essai gratuit d'odoo.com : vous arrivez directement sur la version **19.0**.

LE LIEN

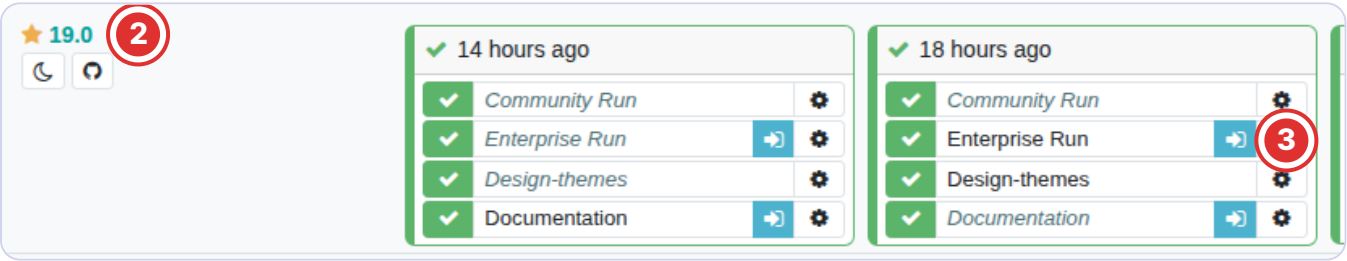
https://runbot.odoo.com

1. **Ouvrez Runbot et repérez la barre de recherche**
   Elle se trouve en haut à droite de la page.

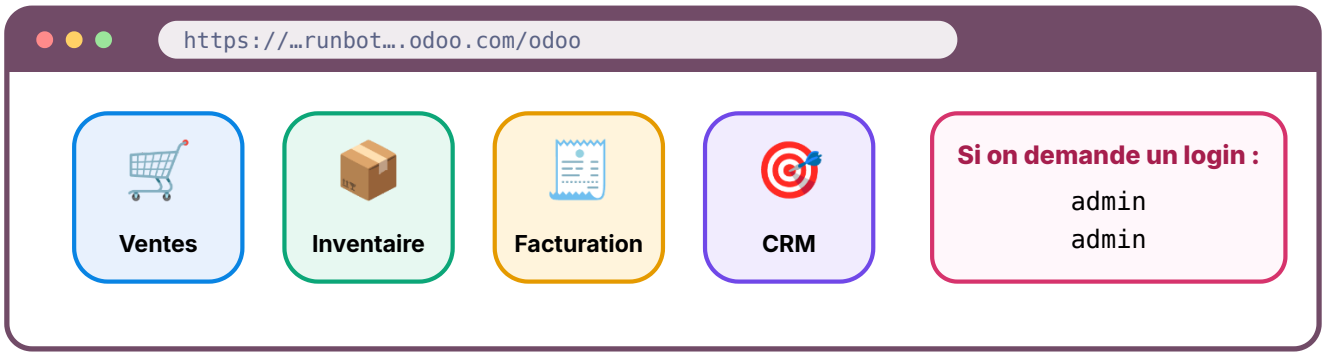


2. **Tapez 19.0 puis Entrée, et choisissez la ligne « 19.0 »**
   Prenez la ligne qui s'appelle **exactement** « 19.0 » (la première, avec l'étoile). Ignorez les lignes aux noms très longs : ce sont des travaux en cours.

3. **Cliquez sur une icône bleue de connexion**
   Cherchez l'icône bleue à droite de « Community Run ». Si cette ligne n'en a pas (comme ici), utilisez celle de « Enterprise Run » : les deux ouvrent Odoo 19. Essayez une autre colonne si besoin.



4. **Odoo s'ouvre : vous pouvez jouer !**
   Si l'on vous demande de vous connecter, utilisez l'identifiant `admin` et le mot de passe `admin`.



À ESSAYER

① **Applications** : installez Ventes, Inventaire et Facturation.
② Refaites le circuit A1 : devis → confirmer → livraison.
③ **Paramètres → Activer le mode développeur**, puis **Technique → Modèles** : cherchez `sale.order` et ses champs.

ATTENTION

Une instance Runbot est **publique et temporaire** : elle peut disparaître à tout moment. N'y saisissez aucune donnée personnelle.